VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi - 590 018, Karnataka

Capstone Project Phase – 1
(Course code: 24AM6PWPW1)

PROJECT WORK REPORT

AI-Driven Analysis and Optimization of CI/CD Pipelines for Build Failure Prediction

In the Department of

Machine Learning

(UG Program: B.E. in Artificial Intelligence and Machine Learning)

Submitted by

Pakam Yasaswini 1BM23AI121
Reem Rashid 1BM23AI148
Sarayu KS 1BM23AI171

6th Semester

Under the Guidance of

Dr. Rajeshwari M.R.

Designation
Dept. of MEL, BMSCE, Bengaluru – 19

DEPARTMENT OF MACHINE LEARNING

B.M.S. COLLEGE OF ENGINEERING

(An Autonomous Institute, Affiliated to VTU)

P.O. Box No. 1908, Bull Temple Road, Bengaluru - 560 019

B.M.S COLLEGE OF ENGINEERING

(An Autonomous Institute, Affiliated to VTU)
P.O. Box No. 1908, Bull Temple Road, Bengaluru - 560 019

DEPARTMENT OF MACHINE LEARNING

CERTIFICATE

This is to certify that Ms. Pakam Yasaswini bearing USN: 1BM23AI121, Ms. Reem Rashid bearing USN: 1BM23AI148, and Ms. Sarayu K S bearing USN: 1BM23AI171 have satisfactorily completed the Course – Capstone Project Work (Course Code: 24AM6PWPW1) with the title **"AI-Driven Analysis and Optimization of CI/CD Pipelines for Build Failure Prediction"** in partial fulfillment of academic curriculum requirements of the 6th semester UG Program – B. E. in Artificial Intelligence and Machine Learning in the Department of Machine Learning, BMSCE, an Autonomous Institute, affiliated to Visvesvaraya Technological University, Belagavi during May 2026. It is also stated that the base work considered for the presentation is used only for academic purpose and not used in its original form anywhere for award of any degree.

Student Signature

Signature of the Faculty,

Dr. Rajeshwari M.R.
Designation, Dept. of MEL, BMSCE

Signature of the Head

Dr. M Dakshayini
Prof. & Head, Dept. of MEL, BMSCE

External Examination

Examiner Name and Signature

1.

2.

DECLARATION

We, **Ms. Pakam Yasaswini** bearing USN: 1BM23AI121, **Ms. Reem Rashi**d bearing USN: 1BM23AI148, and **Ms. Sarayu K S** bearing USN: 1BM23AI171, hereby declare that the project titled "AI-Driven Analysis and Optimization of CI/CD Pipelines for Build Failure Prediction" is an original work carried out by us under the guidance of Dr. Rajeshwari M.R., Assistant Professor, at BMSCE, Bengaluru. The work presented in this report has not been submitted elsewhere for any other degree, diploma, or certification.

We declare that all the references and sources used in the report have been properly acknowledged and cited. No part of this report has been copied, plagiarized, or reproduced from another work without proper attribution.

Pakam Yasaswini -(USN: 1BM23AI121)
Reem Rashid - (USN: 1BM23AI148)
Sarayu KS - (USN: 1BM23AI171)

ACKNOWLEDGEMENT

We would like to express our deepest gratitude and sincere thanks to our guide **Dr. Rajeshwari M.R.**, Assistant Professor, Department of Machine Learning, BMSCE, Bengaluru, for her keen interest, continuous guidance, and encouragement in bringing this project work to completion.

We are grateful to **Dr. M. Dakshayini**, Professor and Head, Department of Machine Learning, Bengaluru, for her valuable suggestions and support throughout the project work.

We express our profound gratitude to the **Hon'ble Management and Dr. Bheemsha Arya**, Principal, BMSCE, Bengaluru, for providing the necessary facilities and a supportive environment to carry out this project successfully.

We would also like to thank all the staff members of the Department of Machine Learning for their support, encouragement, and guidance during the course of this project.

Last but not the least, we would like to thank our parents, family, and friends whose constant encouragement and support made this work possible.

Pakam Yasaswini -(USN: 1BM23AI121)
Reem Rashid - (USN: 1BM23AI148)
Sarayu KS - (USN: 1BM23AI171)

ABSTRACT

Continuous integration and continuous deployment (CI/CD) pipelines have become crucial elements of modern software development, facilitating the automated process of software development, testing, and deployment. While CI/CD pipelines are vital to improving software development speed, maintaining software quality, and reducing the efforts of software developers, frequent build failure issues, dependency conflicts, flaky tests, and lengthy execution time have become obstacles to increasing developer productivity and deploying software quickly. Current CI/CD tools and services have been designed for automating software pipelines and monitoring their execution; there is little intelligent assistance in failure prediction and pipeline optimization.

The current study describes a machine learning-based approach to intelligently analyzing and optimizing CI/CD pipelines. The presented tool extracts historical information about pipeline executions and related failures from popular CI/CD solutions like Jenkins and GitHub Actions. The data is preprocessed and analyzed to detect patterns related to pipeline failure and inefficiency.

ML-based models like Random Forest, Decision Tree, and Support Vector Machine are used to predict CI/CD build failures even before the build completes its execution. Additionally, the system is designed to use anomaly detection methodologies to detect any slow or unusual CI/CD pipeline steps and apply log similarity-based analysis to automatically diagnose the root cause of the problem.

The suggested solution involves a visualization dashboard that highlights failure trends, bottleneck detection, prediction of the build process, and optimization suggestions in a user-friendly way. With the implementation of predictive analysis, anomaly detection, and intelligent monitoring within the CI/CD process, the system strives to increase the reliability of the pipeline and prevent unnecessary wastage of resources.

TABLE OF CONTENTS

Declaration		**i**
Acknowledgement		**ii**
Abstract		**iii**
Table of Contents		**iv**
1	**INTRODUCTION**	
	Overview	
	Organization of the Report	
2	**PROBLEM STATEMENT**	
	Problem Statement	
	Motivation	
	Objectives and Scope	
3	**LITERATURE SURVEY**	
	3.1 Existing System	
	3.2 Research Gap	
4	**SYSTEM REQUIREMENT SPECIFICATION**	
	Functional Requirements	
	Non-functional Requirements	
	Hardware Requirements	
	Software Requirements	
5	**SYSTEM DESIGN**	
	System Architecture	
	Proposed Methodology	
	Detailed Design	

	5.3.1 Class Diagram **5.3.2** Activity Diagram **5.3.3** Use Case Diagram	
6	**IDENTIFICATION OF TOOLS AND TECHNIQUES**	
7	**REFERENCES**	

LIST OF FIGURES

Figure No.	Description
Fig 1	Overview of Proposed AI-Driven CI/CD Optimization System
Fig 2	System Architecture of the Proposed CI/CD Optimization Framework
Fig 3	Proposed Methodology for AI-Driven CI/CD Pipeline Optimization
Fig 4	Class Diagram of the CI/CD Optimization System
Fig 5	Activity Diagram of the CI/CD Optimization System
Fig 6	Use Case Diagram of the CI/CD Optimization System

LIST OF TABLES

Table No.	Description
Table 3.1	Literature Survey Summary
Table 4.1	Hardware Requirements
Table 4.2	Software Requirements

INSTRUCTIONS

1. **Abstrac**t: (**1-page)** A brief summary that emphasizes the key aspects, explains its importance, and outlines the research methods and findings. It should be concise, self-sufficient, and capable of serving as a standalone overview of the project.
2. **Introduction: (3-4 pages)** Provide an introductory paragraph that explains the context and importance of your project. State the problem statement or research question your project aims to address.
    a. **Organization of the Report**
3. **Literature Survey: (3-4 pages) Minimum of 30 relevant journals/conferences.** Summarize relevant background information, research, and existing solutions related to your project. Highlight key studies, theories, or technologies that are relevant to your project.
    a. **Existing Systems:** the current solution or framework available that addresses the problem being studied or improved upon. It involves an overview of the problem/idea, methods, tools, or technologies that are already in place, their strengths, weaknesses, and how they are applied in practice.
    b. Research Gap
4. **Problem statement: 1-paragraph**
5. **Objective and Scope: 1 paragraph of scope and 3 objectives** This should give final outcome of the project. Objectives should be clearly specified. What will be the positive impact of the project on society/academics/industry should be specified.
6. **Proposed Methodology: 1-2 pages** should consist of the steps to be followed to achieve the objective of the project during the project development. Describe the methods, tools, and techniques you plan to use to accomplish the project objectives. Explain the approach you will take to gather data, conduct experiments, or perform analysis.
7. **References** specify the description of the study material referred to, that include the list of books, website links (complete URLs) and research papers.

Guidelines for preparation of VI Semester Capstone Project work report

- Font Type: **Times New Roman**
- Chapter Name: 14 pt. all caps
- First level title font (Sections): 14 pt. Bold

  Example: For Chapter 2, First level titles will be 2.1, 2.2, etc
- Second level title font (Subsections): 12 pt. Bold

  Example: For Chapter 2, Second level titles will be 2.1.1, 2.1.2 etc & 2.2.1,2.2.2 etc
- Body Text font: 12 pt. Normal
- Line Spaces: 1.5

CHAPTER 1 – INTRODUCTION

1.1 Overview

Introduction to CI/CD Pipelines

The implementation of Continuous Integration and Continuous Deployment (CI/CD) pipelines has become crucial in the modern era of software development and DevOps engineering. Modern software development requires rapid application delivery without compromising the software quality and reliability. Traditional ways of software integration and testing can no longer cope with the increasing complexity and size of contemporary applications. For this reason, CI/CD pipelines have been widely adopted in software development to automate software integration, testing, and deployment.

Continuous Integration is a stage when developers continuously make changes in code, which then get incorporated into a shared repository. Whenever there are any updates to the code, automated builds are performed along with software testing to ensure its correctness. In addition, at the Continuous Delivery stage, changes that have passed software testing are automatically deployed to the staging or production environment.

Existing CI/CD Platforms

A variety of CI/CD tools including Jenkins, GitHub Actions, GitLab CI/CD, Travis CI, CircleCI, and Azure DevOps are extensively used in the software development industry. The tools automate various stages of the software development life cycle such as source code integration, dependency management, building, testing, packaging, deployment, and monitoring. Various pipelines have multiple stages that can be executed serially or concurrently depending upon project needs. Automation offered by these tools facilitates early detection of problems and improves the process of deployment.

Such tools offer visualization capabilities in the form of dashboards and logs, along with monitoring that allows developers to handle software delivery processes more effectively.

Challenges in Existing CI/CD Systems

Despite all the benefits, modern CI/CD pipeline technology still faces certain difficulties and issues. Research works analyzing millions of CI/CD runs suggest that nearly 20-40% of all

software builds fail because of some reason – from testing problems, dependency conflicts, wrong configurations to compilation errors, flaky tests, and unstable execution environments. Build failures disrupt development processes, cause delays in releasing software applications, and impact development productivity negatively.

The problem related to the manual labor needed for debugging failed builds is one of the most prominent problems in relation to CI/CD pipeline operation. To fix build failures, a developer needs to analyze numerous execution logs in order to find out what went wrong. Given that pipelines usually have several stages and many other components, analyzing build failures can take lots of time. Research shows that developers spend about 30-60 minutes on each failed build. This issue adds up developer workload.

Apart from builds failures, CI/CD pipelines experience performance inefficiencies like excessive execution time, testing redundancy, stage scheduling problems, and wasted resources. Some stages can take unacceptably long time to finish due to dependency installation issues, testing problems, and infrastructural issues.

Role of Artificial Intelligence and Machine Learning

With the development in the fields of Artificial Intelligence (AI), Machine Learning (ML), and Data Analytics, there is scope for enhancement in the process of CI/CD pipeline management through predictive and prescriptive approaches. Machine learning algorithms will help understand the past build data to recognize any patterns that occur due to failed builds and anomalous behavior in the pipelines. These will help developers predict the issues and prevent failures before they occur within the pipeline.

A number of research papers use machine learning techniques such as Random Forest, Decision Trees, Support Vector Machine (SVM), Neural Network, and Deep Learning models to predict the failures in CI/CD builds. Some others consider anomaly detection, automated logging, and clustering. However, all these studies individually solve some specific issues but none integrate them in an intelligent manner to offer a solution for complete pipeline analysis.

Proposed System Overview

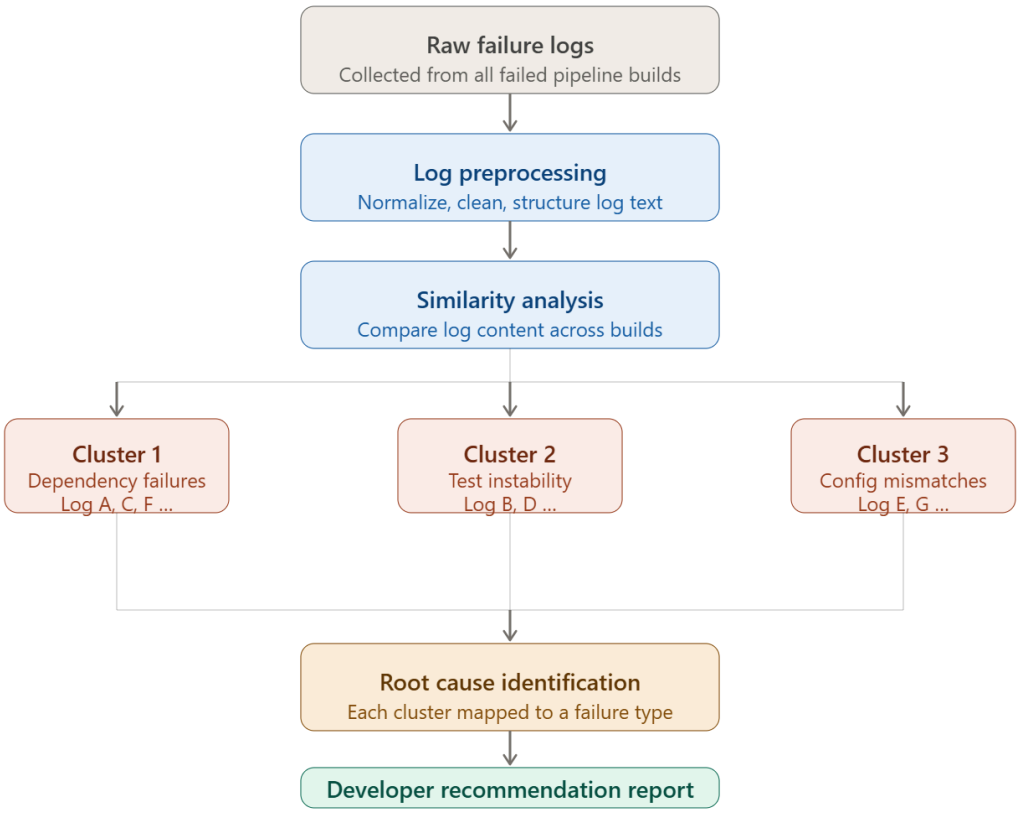


Fig 1: Overview of proposed AI-driven CI/CD optimization system

The proposed research work on the project "AI-Driven Analysis and Optimization of CI/CD Pipelines for Build Failure Prediction" will help address the above-discussed constraints through the development of an intelligent solution for predicting pipeline builds and suggesting optimization approaches. Through the implementation of advanced technologies of machine learning, anomaly detection, and log analysis, the system will be able to predict build failures, recognize bottlenecks in processes, detect recurring patterns, and suggest optimization solutions.

The proposed platform will collect various historical data about the CI/CD pipeline, such as logs, tests' results, execution statistics, stage durations, and commits, that will serve as the input for analysis. Such relevant features as build time, code churn, failure frequency, commit rates, and execution patterns will be used as a basis for machine learning models.

For prediction of possible failures in CI/CD processes, the system will use machine learning algorithms, such as Random Forest, Decision Tree, and Support Vector Machine, to make predictions based on the collected historical data about pipeline processes. Furthermore, anomaly detection technology will help identify slow or abnormal stages that influence

pipeline performance negatively. For further investigation, clustering techniques will group similar failure logs together.

Significance of the Project

The suggested platform is aimed at making optimization suggestions for increasing the efficiency of the pipelines. Such suggestions may include parallel processing of independent stages, decreasing redundant testing and optimal allocation of resources. Thus, using the platform can decrease the time needed for executing tasks and make pipelines more reliable. Besides, the system features a visualization dashboard where developers and administrators will be able to observe predictions of the builds, bottleneck problems, trends of failures and suggestions for optimizations in an interactive way.

The current project possesses high importance in the field of software engineering and DevOps. It can increase software production speed and decrease debugging efforts. Intelligent optimization of pipelines can decrease waste of computational resources and increase the quality of software. This project shows that Artificial Intelligence and Machine Learning methods can be utilized in order to enhance CI/CD processes in modern software engineering environments.

1.2 Organization of the Report

This report is organized into seven chapters.

Chapter 1 introduces the project and its significance.

Chapter 2 presents the problem statement, motivation, objectives, and scope.

Chapter 3 reviews existing systems, related research, and research gaps.

Chapter 4 outlines the functional, non-functional, hardware, and software requirements. **Chapter 5** describes the proposed system design, architecture, methodology, and UML diagrams.

Chapter 6 discusses the tools and techniques used for implementation.

Chapter 7 lists the references used in the study.

CHAPTER 2 – PROBLEM STATEMENT

2.1 Problem Statement

The process of software development nowadays relies significantly on the use of CI/CD pipelines that automate processes of software building, testing, and deployment. However, despite being efficient and increasing the level of automation and speed, CI/CD pipelines suffer from numerous failures such as test failures, dependency issues, configuration mismatches, compilation problems, flaky tests, and execution instability. Based on research studies, almost 20-40% of software builds fail, delaying software delivery and decreasing developer productivity. Currently available CI/CD tools like Jenkins, GitHub Actions, and GitLab CI/CD primarily concentrate on execution and observation of processes without any intelligent features of predictive analysis of potential failures, root cause identification, optimization of bottlenecks, and generation of recommendations for further improvement. The developer needs to investigate extensive execution logs and spend time on identifying problems with their pipeline execution. As a result, it is reasonable to develop an artificial intelligence-based solution that will be capable of analyzing CI/CD pipeline execution history, predicting possible failures, detecting abnormalities in CI/CD execution, clustering failure cases, and generating optimization suggestions.

2.2 Motivation

The implementation of CI/CD pipelines is one of the most important aspects in today's DevOps practices because of their ability to automate software delivery processes. While automation increases developer productivity, the problems related to pipeline failures pose significant challenges in software development practice. Frequent failures disrupt development work, consume unnecessary computational resources, and increase delays in delivering new software products.

On average, developers take from 30 minutes to an hour of their time manually analyzing execution logs in order to find errors that cause failures in a pipeline. The process becomes even more complicated in case when pipelines have numerous steps and dependencies between each other. Today's CI/CD services primarily focus on automation and monitoring without offering any intelligent analytics functionality.

This research seeks to explore the possibilities of creating an artificial intelligence-powered CI/CD optimization platform that would be able to predict failures, discover inefficient pipeline steps, analyze patterns of failures, and generate recommendations.

2.3 Objectives and Scope

Objectives

1. Developing a machine learning-based solution that can predict CI/CD pipeline build failures based on historical data and logs.
2. Detecting bottlenecks and abnormal execution behaviors in CI/CD pipeline stages for optimizing the pipeline process.
3. Providing automated root cause analysis and optimizations to minimize developer efforts in debugging and improving reliability.

Scope

In this study, historical data of CI/CD pipeline builds is used to analyze the performance issues and predict pipeline build failure events. Machine learning algorithms will be used for anomaly detection in the data. The analysis will provide insights on build failures and performance issues in the data. The findings will be used to provide predictive analytics, bottleneck detection, root cause analysis, and optimizations for the developers.

CHAPTER 3 – LITERATURE SURVEY

3.1 Existing Systems

Continuous Integration and Continuous Deployment (CI/CD) systems are widely used in modern software development to automate software building, testing, and deployment processes. These systems help development teams improve software delivery speed, reduce manual effort, and maintain continuous integration workflows. Popular CI/CD platforms such as Jenkins, GitHub Actions, GitLab CI/CD, and Travis CI are commonly used across software industries.

Jenkins is one of the most widely used open-source CI/CD automation tools. It supports automated builds, testing, and deployment using customizable pipelines and plugin integration. Jenkins enables efficient automation of software delivery tasks but mainly focuses on execution and monitoring. It lacks intelligent capabilities for predicting failures or automatically identifying root causes.

GitHub Actions is a cloud-based CI/CD platform integrated directly with GitHub repositories. It automates workflows using YAML configuration files and simplifies software integration and deployment processes. Although it improves automation efficiency, developers still need to manually inspect logs whenever failures occur because the platform lacks predictive analytics and optimization features.

GitLab CI/CD provides integrated DevOps functionality including automated testing, deployment, monitoring, and repository management within a single platform. It supports scalable software delivery workflows but mainly focuses on automation and execution management rather than intelligent prediction and optimization.

Travis CI is commonly used in open-source software projects for automated build and testing processes. It supports multiple programming languages and integrates easily with GitHub repositories. However, it provides limited analytical capabilities and requires manual debugging of failed builds.

Monitoring and visualization tools such as Grafana, Kibana, and ELK Stack are also widely used for analyzing CI/CD logs and execution metrics. These tools help developers track pipeline performance and visualize execution trends. However, they mainly provide

monitoring support and rely heavily on manual interpretation for identifying issues and inefficiencies.

Several recent research studies have explored the use of Artificial Intelligence and Machine Learning techniques for improving CI/CD pipeline management and build failure prediction.

C. Zhang et al. (2022) proposed BuildSheriff, a change-aware test failure triage system for continuous integration builds. The system uses stack trace similarity and code change analysis to automatically group related test failures and reduce debugging effort. However, the approach mainly supports Java projects and Travis CI environments and struggles with flaky tests and unstable execution behavior.

I. Saidani et al. (2020) proposed a machine learning approach using Multi-Objective Genetic Programming for predicting CI build failures using historical build information and software metrics. The study achieved good prediction accuracy but mainly focused on prediction and did not provide automated root cause analysis or optimization support.

A. Barrak et al. (2021) investigated major causes of CI build failures using Random Forest models and software development metrics. The research identified code complexity, build history, and developer activity as important failure indicators. However, the system performed poorly on imbalanced datasets and mainly focused on Java-based projects.

T. Rausch et al. (2017) conducted a large-scale empirical study on build failures in open-source Java projects. The study found that test failures contribute to the majority of broken CI builds. Although the research provided useful statistical insights, it did not include predictive or optimization mechanisms.

I. Saidani et al. (2022) applied deep learning models such as LSTM and CNN for CI build failure prediction and achieved improved prediction accuracy compared to traditional machine learning models. However, deep learning approaches required high computational resources and provided limited explainability for predictions.

T. Durieux et al. (2020) analyzed restarted and flaky builds in Travis CI pipelines and studied common causes such as unstable tests and network-related failures. The study focused mainly on flaky builds and did not provide intelligent optimization or prediction mechanisms.

R. Farihane (2025) presented a systematic mapping study on AI-driven CI/CD pipeline optimization techniques. The study highlighted the growing adoption of machine learning

methods for pipeline analytics, failure prediction, and automation. However, the work mainly reviewed existing techniques and did not propose an implementation framework.

R. Kumar et al. (2025) proposed machine learning models such as Support Vector Machine (SVM) for predicting CI/CD build failures using historical pipeline datasets. Although the study demonstrated promising prediction performance, it lacked automated root cause analysis and bottleneck detection capabilities.

Most existing systems mainly focus on automation, monitoring, or individual prediction tasks. Very few systems integrate build failure prediction, anomaly detection, root cause analysis, and optimization recommendation generation within a single intelligent platform.

3.2 Research Gap

Currently, CI/CD tools prioritize automation and monitoring of executions without emphasizing intelligent analytics and optimization. While there are several research papers that suggest the use of machine learning algorithms for detecting failures in pipelines and analyzing logs, most of them cover just a particular part of the whole issue.

There are some systems that deal exclusively with predicting failures; others are designed for log clustering or anomaly detection alone. Very few approaches involve building a tool capable of dealing with all four mentioned tasks simultaneously. Currently used models suffer from issues associated with imbalanced data sets, scalability problems, and lack of explainability.

The majority of research projects are applicable only for certain CI/CD tools, like Travis CI or Jenkins. Furthermore, many systems require active involvement in terms of interpreting failure logs manually.

Thus, there is a need for developing an artificial intelligence-based CI/CD optimization solution.

CHAPTER 4 – SYSTEM REQUIREMENT SPECIFICATION

4.1 Functional Requirements

The recommended artificial intelligence-based pipeline optimization system should cater to the following functional requirements:

- Data collection of historical CI/CD pipeline records that contain logs, performance metrics, and tests results.
- Pipeline log preprocessing and cleansing for effective analysis.
- Feature extraction from historical records that include build time, failure rate, commit rate, and stages performance.
- Training and execution of machine learning algorithms for build failure prediction.
- Anomaly detection for slow or abnormal pipeline stages.
- Categorization of similar failure logs for automated root cause analysis.
- Optimization suggestions to improve the efficiency of pipelines.
- Visualization of pipeline analytics, prediction, and bottlenecks through dashboards.

4.2 Non-functional Requirements

- **Scalability -**The system should efficiently cope with a huge number of CI/CD logs and execution data from different projects and repositories.
- **Reliability -**The platform should consistently produce reliable predictions and results for the analysis and optimization of the pipelines.
- **Performance -**The system should efficiently process logs and produce predictions using minimum execution time.
- **Accuracy -**Machine learning algorithms should be highly precise to reduce false positives and negatives in prediction.
- **Usability -**The dashboard used for monitoring purposes should be user-friendly and intuitive.
- **Maintainability -**The architecture of the system should make maintenance and enhancement processes easier.

4.3 Hardware Requirements

Component	Specification
Processor	Intel i5 / Ryzen 5 or higher
RAM	Minimum 8 GB
Storage	256 GB SSD
GPU (Optional)	Recommended for deep learning models
Operating System	Windows / Linux / macOS

4.4 Software Requirements

Software Component	Purpose
Python	Core programming language
Pandas	Data preprocessing and analysis

NumPy	Numerical computation
Scikit-learn	Machine learning model development
TensorFlow / PyTorch	Deep learning support
Jupyter Notebook	Model training and experimentation
Jenkins	CI/CD pipeline integration
GitHub / GitLab	Repository management
Grafana / Kibana	Visualization and monitoring
Visual Studio Code	Development environment

CHAPTER 5 – SYSTEM DESIGN

5.1 System Architecture

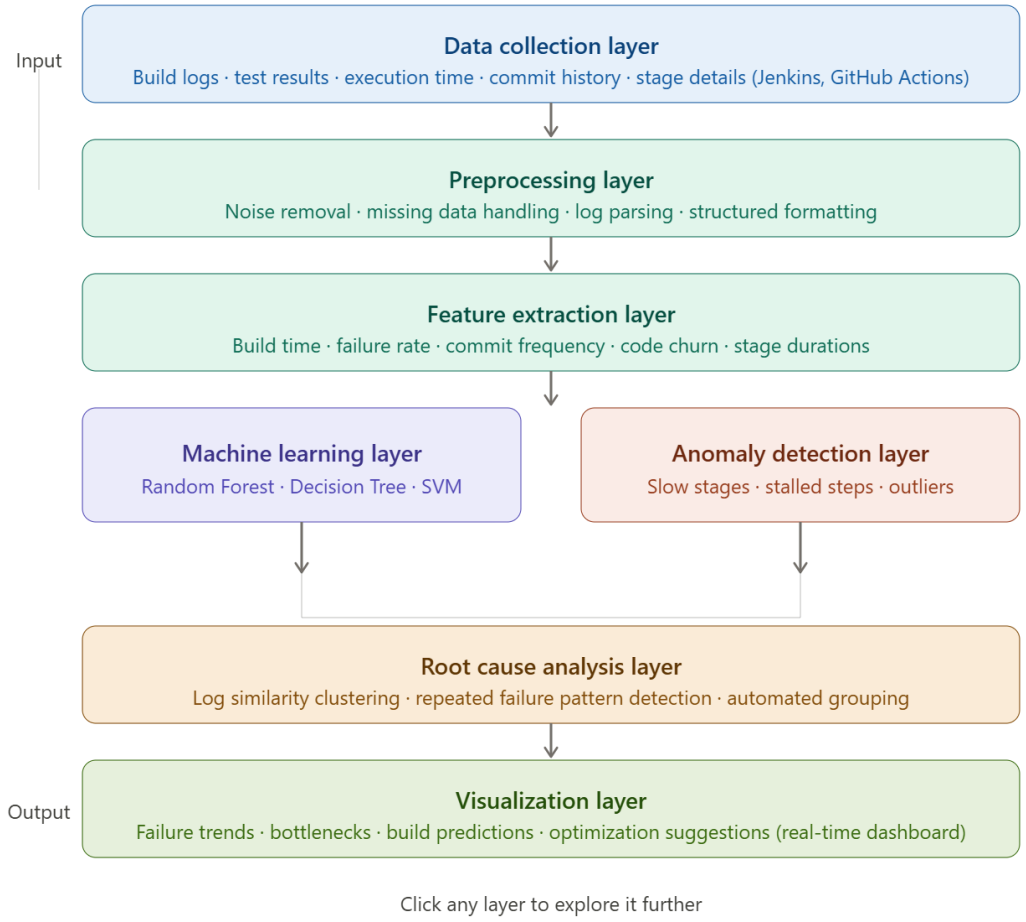


Fig 2: System architecture of the proposed CI/CD optimization framework

The proposed framework has been developed using a modular architecture approach, which can be used for analyzing and optimizing CI/CD pipelines via multiple processing layers.

- **Data Collection Layer:** It will collect historical data about CI/CD pipelines including build logs, test results, time taken for execution, commit history, and details of various stages from CI/CD frameworks such as Jenkins and GitHub Actions.
- **Preprocessing Layer:** This layer will clean and prepare unstructured log files by eliminating noise, dealing with missing data, and preparing them for further analysis.
- **Feature Extraction Layer:** This layer will extract valuable metrics related to CI/CD pipelines including build time, failure rate, commit frequency, code churn, and details about different pipeline stages.

- **Machine Learning Layer:** It will utilize algorithms such as Random Forest, Decision Tree, and Support Vector Machines to predict failure of CI/CD pipelines during execution.
- **Anomaly Detection Layer:** This layer will detect anomalies related to pipeline execution and detect slow/stalled pipeline stages.
- **Root Cause Analysis Layer:** This layer will group similar failure logs and repeated failure events to ease debugging process.
- **Visualization Layer:** This layer will display predictions, bottlenecks, failure trends, and optimization suggestions in real-time via a user-friendly dashboard.

5.2 Proposed Methodology

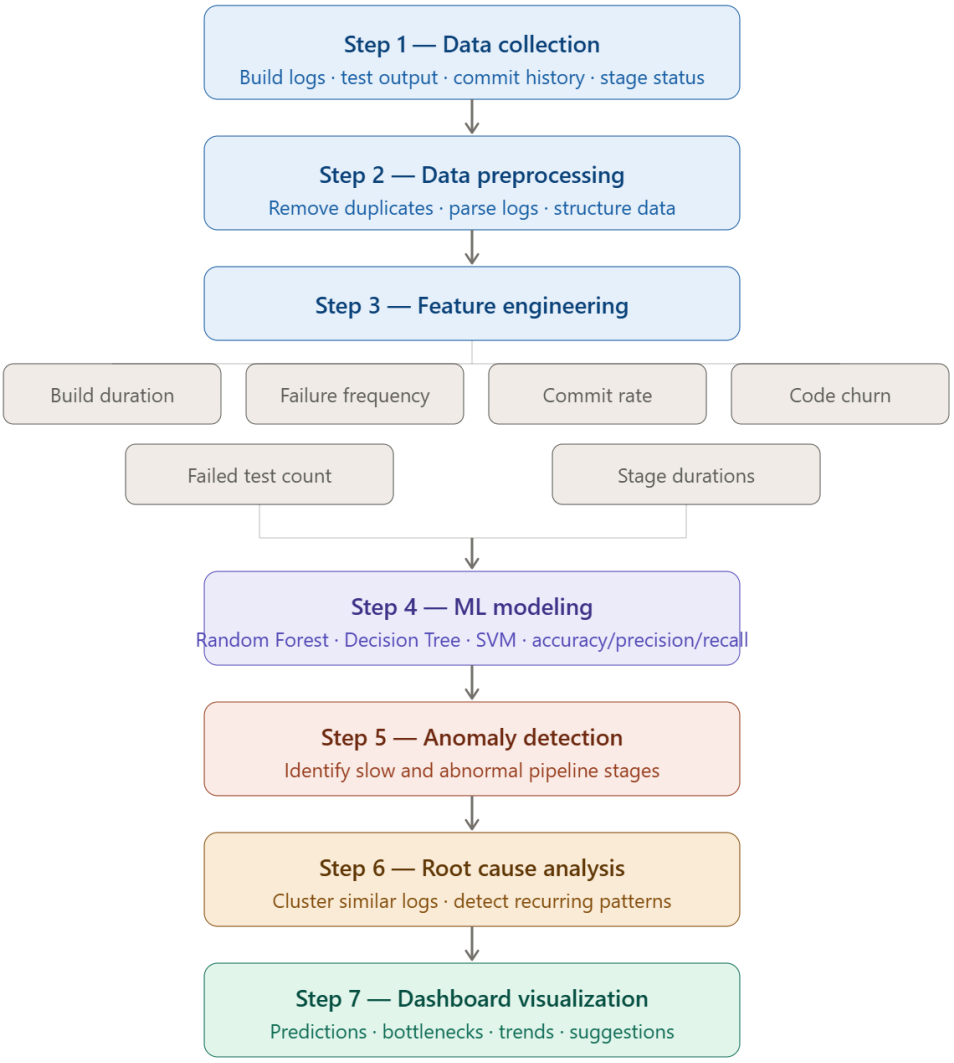


Fig 3: Proposed methodology for AI-driven CI/CD pipeline optimization

The design of the proposed solution employs a methodology-based approach to optimize CI/CD pipelines through machine learning, anomaly detection, and log analysis processes. The methodology revolves around data collection, preprocessing of logs and their feature extraction, training of predictive models, detecting bottlenecks, and visual dashboard recommendations for improvements.

Step 1: Data Collection

Data pertaining to past CI/CD pipelines is acquired from software platforms like Jenkins and GitHub Actions. This data consists of the build logs, testing output, execution time, commit history, stages, and status of builds. Such information will help to analyze pipeline operation and failures.

Step 2: Data Preprocessing

The collected data including logs and execution data is preprocessed to convert it into a structured form. This involves deleting duplicates, missing data, and irrelevant entries from logs. Unstructured data is parsed using log analysis techniques to convert it into machine-readable logs.

Step 3: Feature Engineering

The key features linked to build performance and pipeline operations are extracted from the cleaned dataset and include the following:

- ➔ Duration of build process
- ➔ Frequency of failures
- ➔ Commits rate
- ➔ Rate of code changes
- ➔ Number of failed tests
- ➔ Time taken by each stage in the pipeline

These features allow the recognition of factors that could be related to failure in build processes and pipeline inefficiency.

Step 4: Machine Learning Modeling

The use of machine learning models like Random Forest, Decision Tree, and SVM on historical datasets related to pipelines to classify whether the build process will experience failures. The accuracy of these models is assessed based on metrics like accuracy, precision, and recall.

Step 5: Anomaly Detection

Anomalies within the pipeline are detected through the application of anomaly detection methods. These methods identify pipeline stages with slow and abnormal operations.

Step 6: Root Cause Analysis

Failure logs are categorized through the process of similarity analysis and clustering to detect failure patterns that occur repeatedly. This is done to minimize the effort required by developers to debug and facilitate their failure analysis process.

Step 7: Dashboard Visualizations

Predicted bottlenecks, failure trends, and recommendations are presented through a visualized dashboard. This dashboard serves to help developers analyze the performance of the CI/CD pipeline and behavior during execution.

Through the suggested methodology, the combination of machine learning, anomaly detection, and log analysis provides efficient solutions to the issue of reliable CI/CD pipelines.

5.3 Detailed Design

5.3.1 Class Diagram

The system design provides information about the inner workings and interaction between the different components of the proposed CI/CD optimization system.

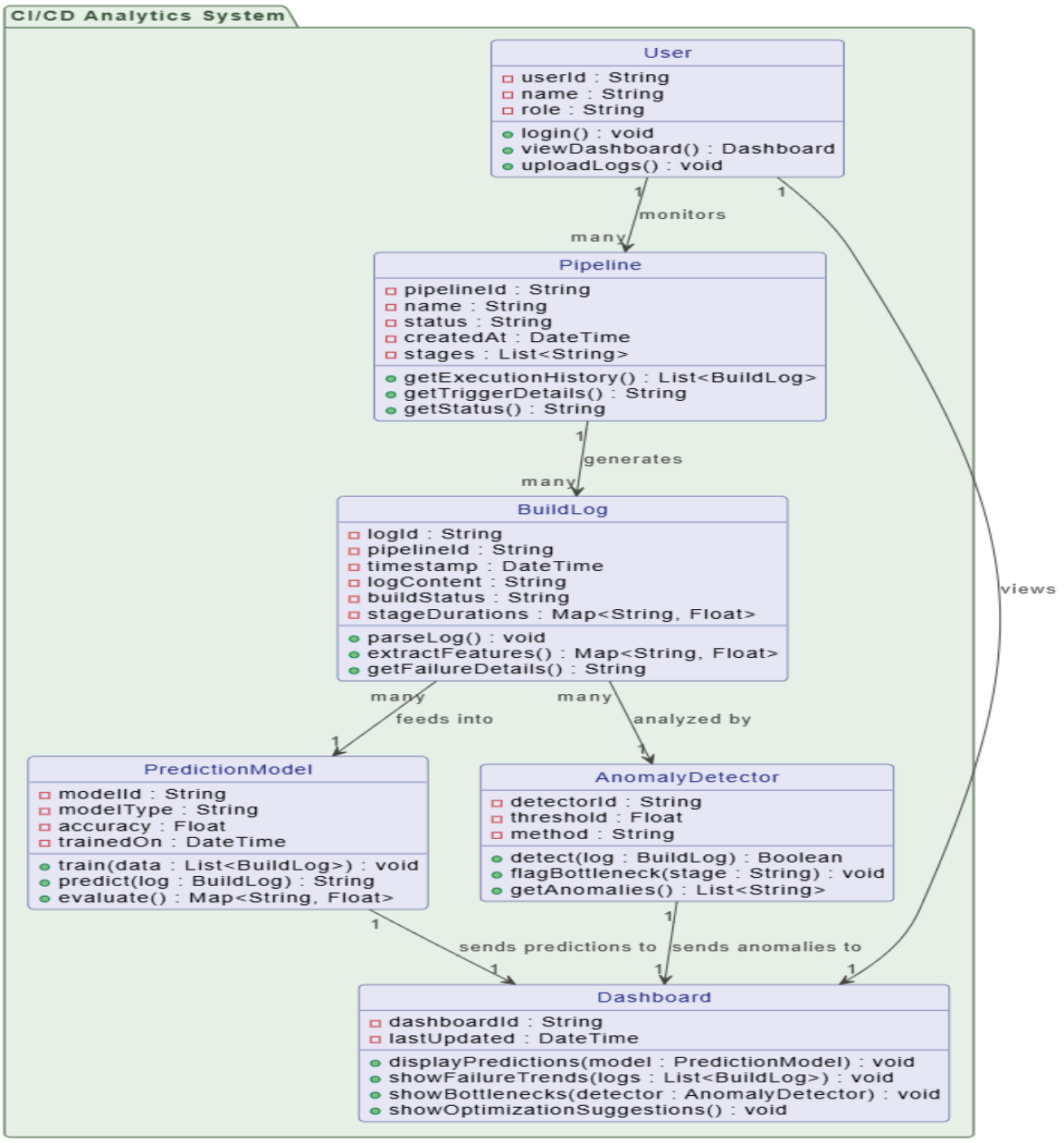


Fig 4: Class diagram of the CI/CD optimization system

The system design provides information about the inner workings and interaction between the different components of the proposed CI/CD optimization system.

The Class diagram shows the main classes and their interactions in the system.

- Classes involved include:
- Pipeline
- BuildLog
- PredictionModel
- AnomalyDetector
- Dashboard
- User

The Pipeline class stores the data regarding the execution of the builds, while the BuildLog class contains data related to logging the process. PredictionModel class performs the task of performing machine learning prediction, and the Dashboard class displays the results and analytics to the user.

5.3.2 Activity Diagram

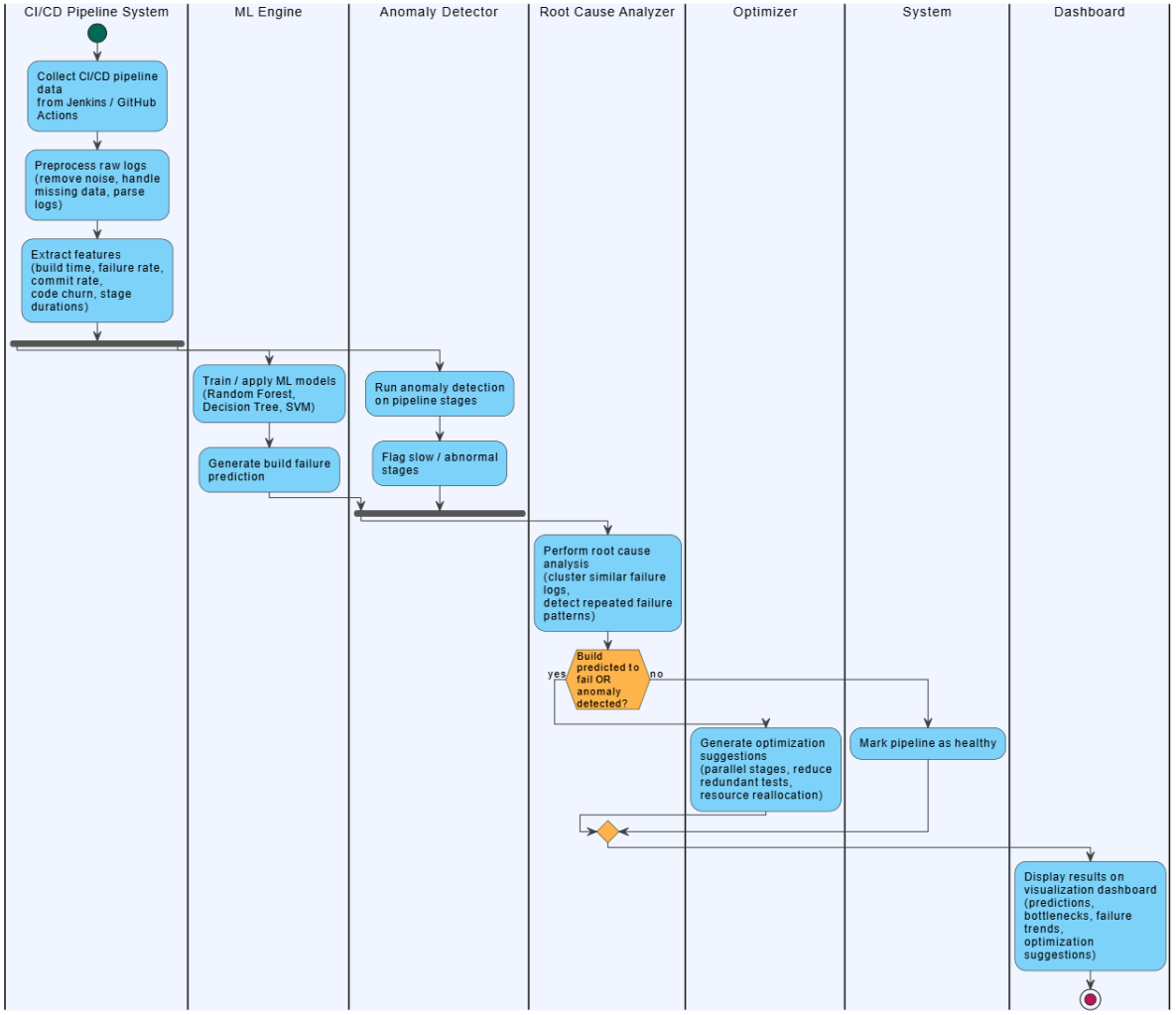


Fig 5:Activity diagram of the CI/CD optimization system

The activity diagram depicts the flow of events in the suggested solution.

The process starts with gathering information regarding the CI/CD pipelines from repositories and build systems. This information will be pre-processed and sent to feature engineering components. Based on the received information, machine learning models produce build failure predictions. Bottlenecks and patterns of build failure occurrence are detected by the anomaly detection and root cause analysis components. Finally, predictions are displayed via the dashboards.

5.3.3 Use Case Diagram

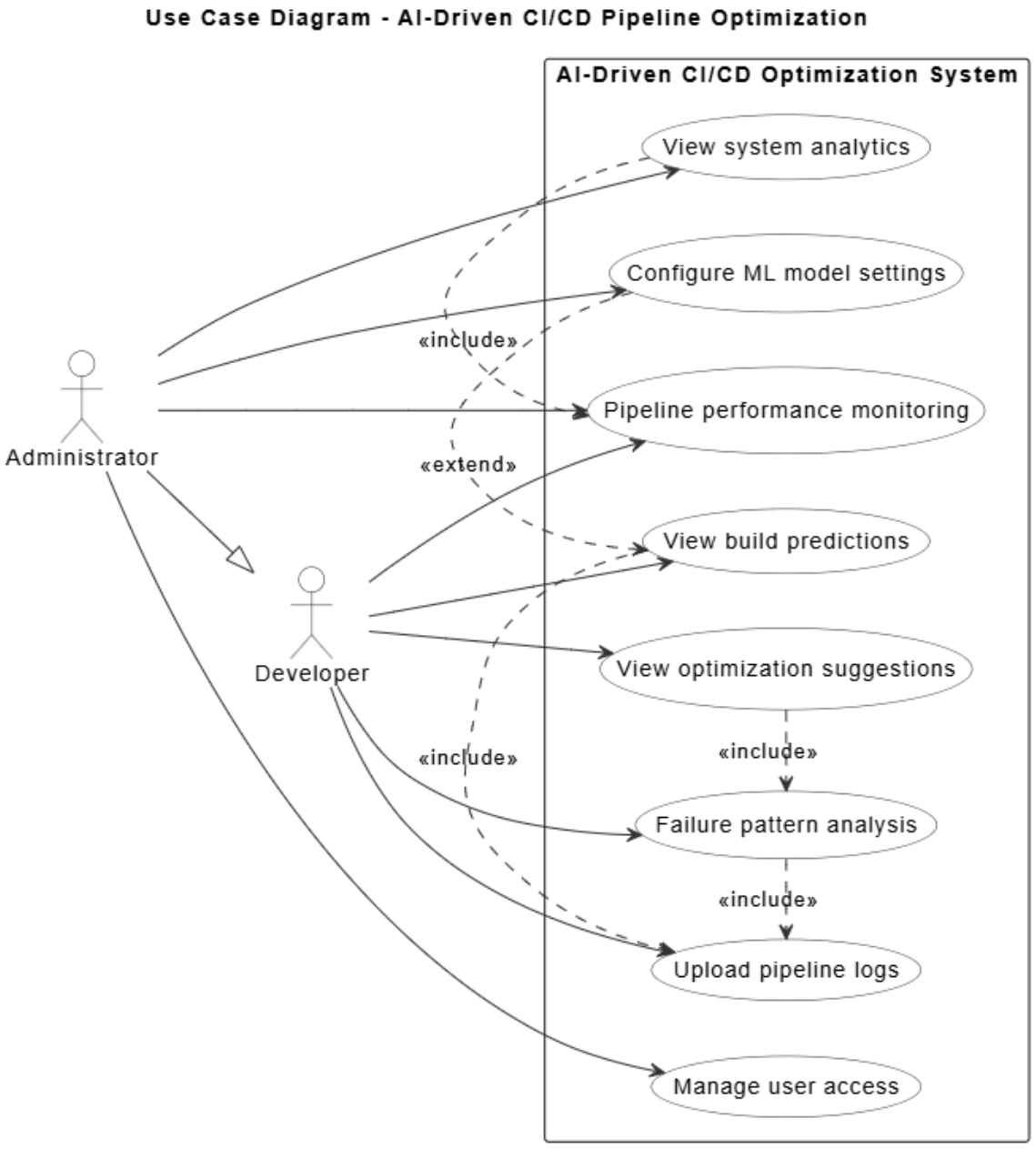


Fig 6:Use case diagram of the CI/CD optimization system

The use case diagram shows user actions in relation to the proposed solution.

Actors:

- Developer
- Administrator

Use Cases:

- Upload pipeline logs
- View build predictions

- Failure pattern analysis
- Pipeline performance monitoring
- View optimization suggestions

CHAPTER 6- IDENTIFICATION OF TOOLS AND TECHNIQUES

6.1 Development Tools

The proposed AI-powered CI/CD optimization system utilizes a combination of tools and technologies selected for their scalability, compatibility, and support for machine learning, data analysis, and pipeline integration.

Python serves as the primary programming language due to its simplicity, cross-platform support, and extensive ecosystem of libraries for machine learning and data processing.

Scikit-learn is used to implement machine learning models such as Random Forest, Decision Tree, and Support Vector Machine (SVM) for build failure prediction and performance evaluation using metrics like accuracy, precision, recall, and F1-score.

Pandas and **NumPy** are used for data preprocessing, cleaning, feature extraction, and numerical computations required for model training and analysis.

TensorFlow and **PyTorch** provide support for future enhancements involving deep learning-based failure prediction and sequence modeling.

Jupyter Notebook is used for data exploration, experimentation, and model development, while **Visual Studio Code (VS Code)** serves as the primary development environment for coding, debugging, and project management.

6.2 CI/CD and Integration Tools

Jenkins is one of the commonly used open source CI/CD automation tools in the software industry. It enables automated builds, test and deployment through customizable pipeline configurations and plugin integrations. The proposed system considers Jenkins as one of the key data sources that provides data regarding the past execution of pipelines, pipeline build logs, pipeline test results, pipeline stage durations, and pipeline build status for analysis and model training.

GitHub Actions is a cloud-based CI/CD platform that is tightly coupled with GitHub repositories. It automates software workflows with the help of YAML-based configuration files and software integration and deployment is made easy. The proposed system uses GitHub Actions as an extra data source to gather information about the pipeline executions and its history for training and testing the machine learning models.

Version Control and **Repository Management Systems** for source code and development changes are GitHub and GitLab. Activity data on these platforms and the frequency of commits, along with the amount of churn in code and repository activity, are important features that can be used to train the machine learning models in the proposed system.

6.3 Visualization and Monitoring Tools.

Grafana is an open source tool for analytics and visualization, which is widely adopted to build dashboards for visualizing interactive and real-time data. The suggested system will use

Grafana to visualise pipeline analytics, failure prediction models, bottlenecks, failure trends, and optimisation recommendations via an easy-to-use, interactive dashboard that is accessible to developers and administrators.

Kibana, which is part of the ELK Stack (Elasticsearch, Logstash, Kibana), is a visualization tool for use with Elasticsearch and Logstash to process and analyze large amounts of logs. In the proposed system, Kibana is leveraged to visualize the logs generated when the CI/CD is executed, to track the performance of the pipelines over the time and as a form of additional monitoring support to complement the machine learning-based analysis performed by the proposed system.

6.4 Techniques Used

Random Forest:
Random Forest is an ensemble machine learning algorithm that combines multiple decision trees to improve prediction accuracy. In the proposed system, it is used as the primary model for CI/CD build failure prediction by analyzing features such as build duration, failure frequency, commit activity, and stage execution times. Its robustness against overfitting and ability to handle complex datasets make it well suited for this task.

Decision Tree:
Decision Tree is a supervised learning algorithm that classifies data using a hierarchy of decision rules derived from input features. It provides interpretable and explainable results, enabling developers and administrators to understand the key factors contributing to pipeline failures and performance issues.

Support Vector Machine (SVM):
Support Vector Machine is a supervised classification technique that identifies the optimal boundary between different classes in a high-dimensional feature space. In this project, SVM is used as an alternative model for build failure prediction and is evaluated against Random Forest and Decision Tree using performance metrics such as accuracy, precision, recall, and F1-score.

Anomaly Detection:
Anomaly detection techniques are employed to identify slow, stalled, or abnormal pipeline stages. These methods help detect performance bottlenecks and unusual execution patterns, enabling early intervention and improving overall pipeline efficiency.

Log Clustering:
Log clustering groups similar failure logs based on textual similarity and recurring patterns. This technique supports automated root cause analysis by categorizing related failures, reducing manual debugging effort, and helping developers identify common issues more efficiently.

Feature Engineering:
Feature engineering involves extracting and selecting meaningful attributes from raw CI/CD pipeline data. Features such as build duration, failure frequency, commit rate, code churn, failed test count, and stage execution times are used to train machine learning models and improve prediction accuracy.

CHAPTER 7- REFERENCES

[1] C. Zhang, B. Chen, X. Peng, and W. Zhao / BuildSheriff: Change-Aware Test Failure Triage for Continuous Integration Builds / 2022 (chenbihuan.github.io/paper/icse22-zhang-buildsheriff.pdf)

[2] I. Saidani, A. Ouni, M. Chouchen, and M. W. Mkaouer / Predicting Continuous Integration Build Failures Using Evolutionary Search / 2020 (ouniali.github.io/papers/IST2020-saidani.pdf)

[3] A. Barrak, E. Eghan, B. Adams, and F. Khomh / Why Do Builds Fail? A Conceptual Replication Study / 2021 (sciencedirect.com/science/article/abs/pii/S0164121221000364)

[4] T. Rausch, W. Hummer, P. Leitner, and S. Schulte / An Empirical Analysis of Build Failures in the Continuous Integration Workflows of Java-Based Open-Source Software / 2017 (ieeexplore.ieee.org/document/7962384)

[5] T. Durieux, C. Le Goues, M. Hilton, and R. Abreu / Empirical Study of Restarted and Flaky Builds on Travis CI / 2020 (2020.msrconf.org/details/msr-2020-papers/21)

[6] T. Ghaleb, D. Costa, and Y. Zou / An Empirical Study of the Long Duration of Continuous Integration Builds / 2019 (link.springer.com/article/10.1007/s10664-019-09695-9)

[7] I. Saidani, A. Ouni, and M. W. Mkaouer / Improving the Prediction of Continuous Integration Build Failures Using Deep Learning / 2022 (researchgate.net/publication/357990835)

[8] R. Farihane, *"CI/CD Pipeline Optimization Using AI: A Systematic Mapping Study,"* MDPI Proceedings, 2025.

[9] R. Kumar, D. N. D. Kollipara, A. H. Mohammed, S. Kumar, F. B. Mohammed, and A. Kumar, *"Optimizing CI/CD Pipelines with AI-Driven Build Failure Prediction: An Empirical Study on Machine Learning Models for Early Failure Detection,"*

[10] Y. Hong, C. Tantithamthavorn, J. Pasuksmit, P. Thongtanunam, A. Friedman, X. Zhao, and A. Krasikov, *"Practitioners' Challenges and Perceptions of CI Build Failure Predictions at Atlassian,"* arXiv preprint arXiv:2402.09651, 2024.

CONCLUSION AND FUTURE WORK

Conclusion

Integration, testing, and deployment of code through CI/CD have been made possible through automation. Despite this, continuous build failures, lengthy execution periods, dependency issues, and manual debuggings remain a challenge among others. Current CI/CD systems concentrate on automations and monitoring but lack advanced intelligent capabilities such as failure predictions and optimizations.

The presented research seeks to enhance the current state of art by designing an AI-based optimization approach to be incorporated in CI/CD systems. The new design involves using machine learning to conduct analysis, log analysis and anomaly detection, which are used to predict failures and detect inefficiencies in pipelines. The platform is then expected to categorize errors, find bottlenecks, detect similar failure patterns, and suggest optimization methods.

In addition, a visualization dashboard will be designed to help in analyzing the efficiency and performance of pipelines in different aspects.

Future Work

While the proposed model exhibits efficient capabilities of intelligent analysis and optimization of CI/CD pipelines, various advancements could be considered in the future for the benefit of the system's performance and scalability.

- Integration of the system with cloud-based platforms like Docker and Kubernetes for large-scale implementation purposes.
- Incorporation of deep learning algorithms for accurate prediction of pipeline data.
- Real-time tracking of pipeline execution and live anomaly detection.
- Compatibility of other CI/CD pipelines other than Jenkins and GitHub Actions.
- Deployment of automated self-repair processes to address common pipeline errors.
- Introduction of advanced analytical tools in the visualization dashboard.
- Utilization of natural language processing techniques in log analysis and root cause identification.

These proposed modifications can significantly enhance the proposed CI/CD optimization tool in terms of efficiency and effectiveness.